پاسخ سوال یک:

الف) اثبات O(n) برای ساخت هرم

اگرچه down_bubble برای هر گره حداکثر O(log n) است، اما تعداد گره‌هایی که در عمق زیاد قرار دارند بسیار بیشتر از گره‌های نزدیک ریشه است. در هرمی با n عضو:

- تعداد گره‌ها در ارتفاع h (فاصله تا برگ‌ها) حداکثر n / 2^(h+1) است.

- هزینه‌ی down_bubble برای گره در ارتفاع h، از O(h) است.

هزینه‌ی کل = مجموع روی h از 0 تا log n از:
(n / 2^(h+1)) × O(h) = O(n) × مجموع (h / 2^h)

از آنجایی که مجموع h/2^h از h=0 تا بی‌نهایت برابر 2 است (یک سری همگرا)، داریم: O(n) × O(1) = O(n)

بنابراین ساخت هرم واقعاً O(n) است، چون اکثریت قریب‌به‌اتفاق گره‌ها (نیمی از کل) در پایین‌ترین سطح هستند و اصلاً نیازی به down_bubble ندارند یا هزینه‌شان ناچیز است.

ب) درخت تصمیم برای n=4

برای n=4 داریم n! = 24 حالت ممکن. طبق لم درخت دودویی، یک درخت با ارتفاع h حداکثر h^2 برگ دارد. برای این‌که همه‌ی 24 حالت جدا شوند نیاز داریم:

$h \geq 24 \Rightarrow h \geq \lceil \log_2 24 \rceil = \lceil 4.58 \rceil = 5^2$

یعنی حتی در بهترین طراحی درخت تصمیم، حداقل ۵ سطح مقایسه لازم است تا از میان ۲۴ ترتیب ممکن، دقیقاً یکی باقی بماند. هیچ الگوریتم مقایسه‌ای برای مرتب کردن ۴ عضو با کمتر از ۵ مقایسه (در بدترین حالت) نمی‌تواند درست کار کند.

پاسخ سوال دوم:

الف) محاسبه‌ی p۱۵ و مقایسه با p۱۲

p15 = 2 / (5 - 1 + 1) = 2/5 = 0.4

p12 = 2 / (2 - 1 + 1) = 2/2 = 1

p12 = 1 چون z1 و z2 عناصر مجاورند: هرکدام که اول به عنوان محور انتخاب شود، دیگری حتماً باید با آن مقایسه شود تا معلوم شود در کدام بخش قرار می‌گیرد.

اما z1 و z5 در دو سر دنباله‌اند و فاصله‌ی زیادی (۳ عضو میانی) بین آن‌ها هست. برای این‌که این دو با هم مقایسه شوند،

باید یکی از آن‌ها اولین عضوی باشد که از میان کل مجموعه‌ی {z1,...,z5} به عنوان محور انتخاب می‌شود. هرچه فاصله‌ی j-i بیشتر شود، احتمال این‌که یکی از عناصر میانی زودتر انتخاب شده و آن‌ها را از هم جدا کند بیشتر می‌شود، پس pij کاهش می‌یابد.

ب) چرا انتخاب تصادفی محور تحلیل را مستقل از ورودی می‌کند؟

فرمول pij = 2/(j-i+1) از این فرض به دست آمد که هر عضو از زیرآرایه‌ی فعلی با احتمال یکسان ممکن است به عنوان محور انتخاب شود. اگر محور را همیشه ثابت (مثلاً آخرین عضو) انتخاب کنیم، این احتمال دیگر یکسان نیست و به ترتیب اولیه‌ی داده وابسته می‌شود؛ مثلاً در آرایه‌ی از پیش مرتب‌شده این انتخاب باعث افراز کاملاً نامتوازن و بدترین حالت $O(n^2)$ می‌شود.

اما وقتی محور هر بار کاملاً تصادفی از بازه‌ی فعلی انتخاب می‌شود، احتمال انتخاب هر عضو مستقل از مقدار آن یا ترتیب اولیه‌ی آرایه است. در نتیجه فرمول pij برای هر آرایه‌ی ورودی به یک اندازه صادق می‌ماند و $E[C(\sigma)] = O(n \log n)$ تضمینی مستقل از داده‌ی ورودی می‌شود.

پاسخ سوال سوم:

الف) چرا فقط اندیس ریشه در inorder کافی است؟

طبق خاصیت پیمایش inorder:

(زیردرخت چپ ⇐ ریشه ⇐ زیردرخت راست)

وقتی برچسب ریشه را در لیست inorder پیدا می‌کنیم، تمام عناصری که قبل از ریشه آمده‌اند، دقیقاً همان اعضای زیردرخت چپ ریشه هستند — نه بیشتر و نه کمتر.

بنابراین طول این بخش (یعنی root_inorder_index) خودش دقیقاً اندازه‌ی زیردرخت چپ است. لیست preorder فقط برای یافتن برچسب ریشه لازم است؛ برای محاسبه‌ی اندازه، خودِ inorder به‌تنهایی کافی است.

ب) چرا فرض «حداکثر دو فرزند» لازم است؟

در پیمایش inorder، وقتی گره‌ای بیش از دو فرزند دارد
(مثلاً سه فرزند: چپ، میانی، راست)، ترتیب پیمایش فقط می‌گوید همه‌ی این گروه‌ها قبل یا بعد از ریشه ظاهر می‌شوند — اما مرز دقیق بین گروه میانی و گروه راست مشخص نیست.

با داشتن فقط preorder و inorder، فقط می‌دانیم ریشه کجاست و چه چیزهایی قبل/بعد از آن‌اند، اما اطلاعاتی درباره‌ی این‌که کجا زیردرخت اول تمام می‌شود و دومی شروع می‌شود نداریم.

برای درخت دودویی این مشکلی نیست، چون فقط یک تقسیم (چپ/راست) وجود دارد و ریشه خودش مرز را مشخص می‌کند.

پاسخ سوال چهارم:

الف) محدودیت مرتب‌سازی شمارشی

چرا با اعداد بزرگ ناکارآمد می‌شود: مرتب‌سازی شمارشی نیازمند آرایه‌ی کمکی به اندازه‌ی m+1 است (m = بزرگترین عدد). اگر آرایه ۱۰ عنصر داشته باشد اما اعداد بین ۰ تا ۱۰۰,۰۰۰,۰۰۰ باشند، باید آرایه‌ای با بیش از ۱۰۰ میلیون خانه ساخت که هم از نظر حافظه و هم زمان (O(n+m)) که در آن m≫n) بسیار ناکارآمد است.

چرا در سناریوی اسکناس‌ها برتری دارد: در آن‌جا n میلیون‌هاست اما بزرگترین مقدار ممکن m فقط 5000 است. پیچیدگی زمانی O(n+m) ≈ O(n) می‌شود، یعنی خطی و بسیار سریع؛ در حالی‌که مرتب‌سازی درجی در بدترین حالت $O(n^2)$ است که با میلیون‌ها عنصر عملاً غیرقابل اجرا می‌شود.

ب) محاسبه‌ی دقیق مقایسه‌ها در Insertion Sort

حالت A=[1,2,3,4,5] (از پیش مرتب): برای هر k، شرط A[i-1]>item بلافاصله نادرست است،

پس فقط یک مقایسه در هر مرحله انجام می‌شود و تعداد کل مقایسه ها برابر است با:

$$(n-1) = 4 \Rightarrow O(n)$$

حالت B=[5,4,3,2,1] (وارونه): برای kام، item باید دقیقاً k مرحله شیفت شود.

تعداد کل مقایسه‌ها:

$$1 + 2 + 3 + 4 = n(n-1)/2 = 10 \Rightarrow O(n^2)$$

این تفاوت (۴ در برابر ۱۰ مقایسه برای n=5) نشان می‌دهد پیچیدگی این الگوریتم کاملاً به ترتیب اولیه‌ی داده وابسته است.